

Abstractions for Programming

Unit -7

Abstractions for Programming

Defining abstractions for programming in multimedia involves creating layers of simplification that allow developers to interact with complex multimedia systems without needing to understand every underlying detail.

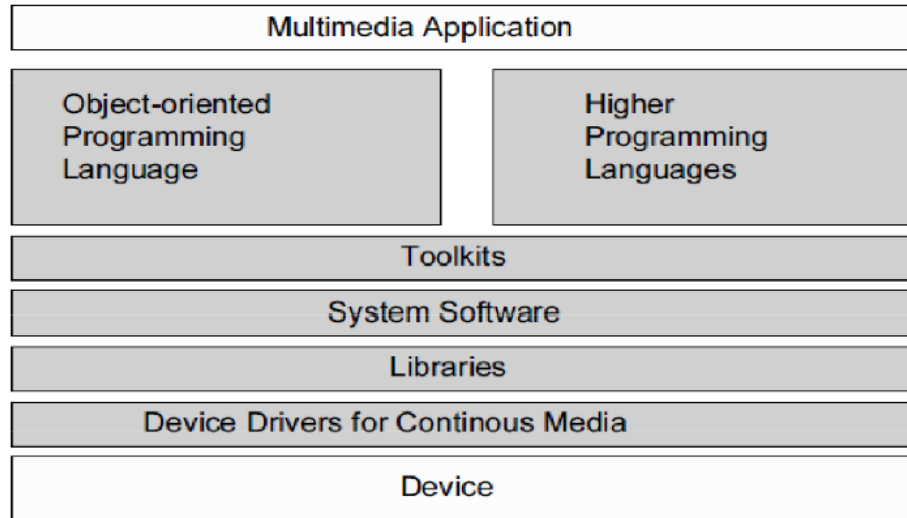
Abstractions for Hardware

The state of the art of programming

- Application code is highly dependent on hardware
- Change of multimedia devices still often requires re-implementation
- Strong hardware dependency may cause problems with:
 - Portability
 - Reusability
 - Coding efficiency

Abstraction Levels

- Common Operating system extensions try to solve this problem
- Different programming possibilities for accessing and representing multimedia data



Abstraction Levels of the Programming of Multimedia Systems

Devices

The lowest level represents the actual hardware components involved in multimedia processing, such as cameras, speakers, displays, and GPUs.

Device Drivers for Continuous Media

- These are software interfaces that allow the operating system to communicate with multimedia hardware.
- Examples include audio drivers, video drivers, and GPU drivers.

Libraries

- A collection of pre-written code that provides functions for multimedia processing.
- Examples: OpenGL (graphics), OpenCV (computer vision), FFmpeg (video/audio processing)

System Software

- The underlying software that manages system resources and provides services for multimedia applications.
- Includes operating systems (Windows, Linux, macOS)

Toolkits

- These are frameworks or SDKs (Software Development Kits) that offer high-level APIs for multimedia application development.
- Examples: GTK, Qt, DirectX, OpenAL (for audio).

Programming Languages

- Provides the application with a class hierarchy for the manipulation of multimedia.
- In the higher programming languages used to implement multimedia applications contains abstractions of multimedia data.

Multimedia Applications

End-user applications that provide multimedia functionality, such as video players, game engines, streaming platforms, and image editing.

Libraries

- Libraries are essential for processing continuous media (audio, video, etc.) because operating systems lack sufficient built-in support for such tasks.
- Libraries provide functions for interacting with hardware, extending GUIs, and passing control instructions to drivers.
- Due to the lack of standardization and OS-level support, there is a wide variety of libraries, each with its own interface and functionality.

Libraries- OpenGL

- 2D and 3D graphics API developed by Silicon Graphics
- Basic idea: “write applications once, deploy across many platforms”
 - PCs
 - Workstations
 - Supercomputers
- Benefits:
 - Stable
 - Reliable and Portable
 - Evolving
 - Scalable (Features like zoom, rectangle....)
 - Well documented and easy to use

Libraries-OpenGL

- Integrated with
 - Windows 95/NT/2000/XP
 - UNIX x Window system

System Software

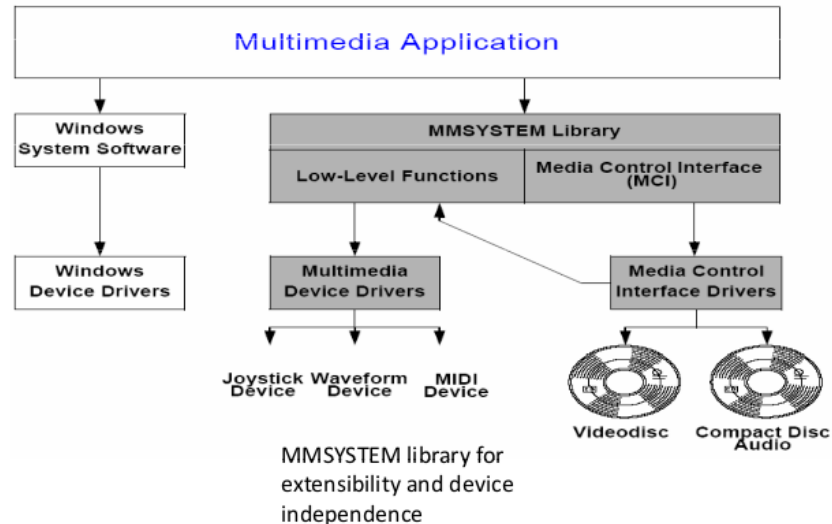
- Device access becomes part of the operating system
- Data as time capsules (file extensions)
 - Each Logical Data Unit (LDU) carries in its time capsule its data type, actual value and life span
 - Useful concept for video, where each frame has a valid lifespan of 40ms
 - Presentation rate is changed for VCR (Video Cassette Recorder) functions like fast forward, slow forward or fast rewind by
 - Changing presentation lifespan of LDUs
 - Skipping of LDUs or repetition of LDUs

System Software

- Data as stream
 - a stream denotes the continuous flow of audio and video data between a source and a sink
 - Prior to the flow the stream is established equivalent to the setup of a connection in a networked environment

System Software – Window MCI

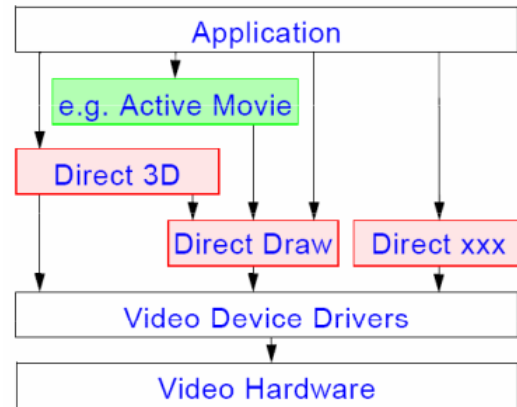
- In Microsoft windows, a media control interface (MCI) provides the interface for processing multimedia data.
- It allows the access to continuous media streams and their corresponding devices.



System Software-DirectX

- Low level APIs and libraries for high performance
- Especially games - formerly known as "Game SDK"
- Direct Access to hardware services
 - E.g: audio & video cards, GPU
 - DirectX = "direct access"
- Strong relationship/interaction

with ActiveX/DCOM



= component of ActiveX



= component of DirectX

System Software-DirectX

- Component : It consists of several APIs, each focused on a specific aspect of multimedia processing:
 - DirectDraw : 2 dimensional graphics capabilities
 - Direct3D : For 3D graphics rendering
 - DirectSound : For audio playback and recording
 - DirectPlay: For network gaming
 - DirectInput: For handling input devices
- Implementation strategy:
 - Hardware Abstraction Layer (HAL)
 - Hardware Emulation Layer (HEL)
 - Media Layer

Toolkits

- **Toolkits** play a crucial role in providing abstraction layers that simplify the development of multimedia applications.
- These toolkits encapsulate complex functionalities (e.g., graphics rendering, audio processing, video playback) into easy-to-use APIs, allowing developers to focus on the application logic rather than low-level details.
- Multimedia toolkits are collections of libraries, APIs, and tools that provide pre-built functionality for handling multimedia tasks.
- Can be embedded into programming languages or object oriented environments

Toolkits

Examples

- SDL (Simple DirectMedia Layer)

A cross-platform toolkit for graphics, audio, and input handling.

- JavaFX

A framework for building multimedia-rich applications in Java.

Toolkits

- **Advantages**

- Ease of use
- Cross-Platform support
- Rapid Development
- Hardware Abstractions

- **Challenges**

- Performance Overhead
- Limited Flexibility
- Learning Curve

Higher Programming Language

- In the higher programming languages, the processing of continuous media data is influenced by a group of similar constructed functions.
- It emphasizes the role of hardware-independent function calls, direct access to multimedia data structures (frame, audio samples) , and communication with real-time processes.

Higher Programming Language

Media can be considered differently inside a programming language

- Media as type
- Media as file
- Media as processes

Media as Types

- Media types in HLLs are data structures or objects that represent different forms of multimedia data (e.g., audio, video, images).
- To provide a high-level abstraction for working with multimedia data, simplifying tasks like playback, editing, and processing.
- Characteristics
 - **Encapsulation:** Media types encapsulate data (e.g., pixel values, audio samples) and operations (e.g., play, pause, resize).
 - **Hardware Abstraction:** Media types abstract the complexities of interacting with hardware (e.g., GPUs, sound cards).
 - **Cross-Platform Support:** Media types are designed to work across different platforms (e.g., Windows, macOS, Linux).

Media as Types

Examples

- Audio:

python

```
import pygame
pygame.init()
sound = pygame.mixer.Sound("audio.wav")
sound.play()
```

- Image:

python

```
from PIL import Image
img = Image.open("image.jpg")
img.show()
```

- Audio:

html

```
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">
  Your browser does not support the audio element.
</audio>
```

Media as File

- instead of considering continuous media as data types they can be considered as files.
- The example treats media streams as file-like objects that can be opened, read, written, mixed, and closed.

```
file_h1 = open(MICROPHONE_1,...)
file_h2 = open(MICROPHONE_2,...)
file_h3 = open(SPEAKER, ...)

...
read(file_h1)
read(file_h2)
mix(file_h3, file_h1, file_h2)
activate(file_h1, file_h2, file_h3)
...
deactivate(file_h1, file_h2, file_h3)
...
rc1 = close(file_h1)
rc2 = close(file_h2)
rc3 = close(file_h3)
```

Media as File

```
file_h1 = open(MICROPHONE 1, ...) -- Open microphone 1 as a file-like object
file_h2 = open(MICROPHONE 2, ...) -- Open microphone 2 as a file-like object
file_h3 = open(SPEAKER, ...)      -- Open speaker as a file-like object

read(file_h1)                     -- Read data from microphone 1
read(file_h2)                     -- Read data from microphone 2

mix(file_h3, file_h1, file_h2)    -- Mix data from microphone 1 and 2, and send to speaker

activate(file_h1, file_h2, file_h3) -- Activate the media streams (start processing)

deactivate(file_h1, file_h2, file_h3) -- Deactivate the media streams (stop processing)

rc1 = close(file_h1)              -- Close microphone 1
rc2 = close(file_h2)              -- Close microphone 2
rc3 = close(file_h3)              -- Close speaker
```

Media as Processes

- The processing of continuous data contains a time-dependency because the life-span of a process equals to the life span of a connection between source and destination.
- A connection can exist locally, as well as remotely.
- Under this consideration, it is possible to map continuous media to processes and to integrate them in an HLL.

Media as Processes

```
PROCESSSS cont_Process_a;
```

```
.....
```

```
On_message_do
```

```
    Set_Volume.....
```

```
    Set_loudness.....
```

```
    .....
```

```
    .....
```

```
    [main]
```

```
pid=create(cont_process_a)
```

```
send(pid, set_volume,3)
```

```
send(pid, set_loudness)
```

```
.....
```

```
PROCESS cont_Process_a; -- Define a process for handling continuous media
```

```
On_message_do           -- Define actions to be taken when a message is received
```

```
    Set_Volume..... -- Set the volume of the media stream
```

```
    Set_loudness..... -- Set the loudness of the media stream
```

```
    .....
```

```
    .....
```

```
    -- Other actions (e.g., pause, stop, adjust quality)
```

```
    -- Additional logic for handling messages
```

```
[main]                -- Main program
```

```
pid = create(cont_process_a) -- Create an instance of the continuous media process
```

```
send(pid, set_volume, 3)    -- Send a message to set the volume to level 3
```

```
send(pid, set_loudness)    -- Send a message to adjust the loudness
```

```
    .....
```

```
    -- Additional commands or logic
```

Programming Language Requirement

The high-level language should support parallel processing, because the processing of continuous data is

- Controlled by the language through pure asynchronous instructions
- An integral part of a program through the identification of media
- must be processed and delivered in real-time to ensure smooth playback and synchronization.

Programming Language Requirement

Different processes must be able to communicate through inter-process communication mechanism which must be able to:

- Understand a priori and/or implicitly specified time requirements (QoS parameters or extracted from the data type)
- Transmit the continuous data according to the requirements
- Initiate the processing of the received continuous process on time

Object –Oriented Approaches

- The object-oriented approach was first introduced as a method for the reduction of complexity in the software development and it is used mainly with this goal today.
- Further, the reuse of software components is a main advantage of this paradigm.
- The basic ideas of object-oriented programming are: **data encapsulation and inheritance**, in connection with class and object definitions.
- The programs are implemented, instead of using functions and data structures, by using classes, objects, and methods.

Class

- A class represents a collection of objects having same characteristic properties that exhibit common behaviour.
- It gives the blueprint or description of the objects that can be created from it.
- Creation of an object as a member of a class is called instantiation. Thus, object is an instance of a class.

Class

- Let us consider a simple class, Circle, that represents the geometrical figure circle in a two-dimensional space. The attributes of this class can be identified as follows –
 - x-coord, to denote x-coordinate of the center
 - y-coord, to denote y-coordinate of the center
 - r, to denote the radius of the circle

Some of its operations can be defined as follows –

- findArea(), method to calculate area
- findCircumference(), method to calculate circumference
- scale(), method to increase or decrease the radius

Object

An object is a real-world element in an object-oriented environment that may have a physical or a conceptual existence. Each object has –

- Identity that distinguishes it from other objects in the system.
- State that determines the characteristic properties of an object as well as the values of the properties that the object holds.
- Behaviour that represents externally visible activities performed by an object in terms of changes in its state.

Objects can be modelled according to the needs of the application. An object may have a physical existence, like a customer, a car, etc.; or an intangible conceptual existence, like a project, a process, etc.

Inheritance

- The concept allows us to inherit or acquire the properties of an existing class (parent class) into a newly created class (child class). It is known as inheritance.
- It provides code reusability.
- Besides, the subclass may add its own attributes and methods and may modify any of the super-class methods.
- From a class Mammal, a number of classes can be derived such as Human, Cat, Dog, Cow, etc.
- Humans, cats, dogs, and cows all have the distinct characteristics of mammals. In addition, each has its own particular characteristics.

Polymorphism

- Polymorphism is originally a Greek word that means the ability to take multiple forms.
- In object-oriented paradigm, polymorphism implies using operations in different ways, depending upon the instance they are operating upon.
- Polymorphism allows objects with different internal structures to have a common external interface.
- Polymorphism is particularly effective while implementing inheritance.

Polymorphism

- Let us consider two classes, Circle and Square, each with a method findArea().
- Though the name and purpose of the methods in the classes are same, the internal implementation, i.e., the procedure calculating area is different for each class.
- When an object of class Circle invokes its findArea() method, the operation finds the area of the circle without any conflict with the findArea() method of the Square class.

Application-Specific Metaphor as Classes

- Multimedia metaphor is a set of user interface visuals, actions and procedures that exploit specific knowledge that users already have of other domains.
- An application-specific class hierarchy introduces abstraction specifically designed for a particular application.
- Thus, it is not necessary to consider other class hierarchies.
- Using this approach, one very easily abandons the actual advantages of object-oriented programming, i.e. the reuse of existing code.

Application-Specific Metaphor as Classes

Example: **KaraokePlayer Class**

Metaphor: A media player specifically designed for karaoke, with features like lyrics display.

Application Specific: This class is not reusable in other applications because it is tailored to karaoke.

```
class KaraokePlayer(MediaPlayer):  
    def display_lyrics(self, lyrics):  
        print(f"Displaying lyrics: {lyrics}")
```

Application–Generic Metaphor as CLasses

- This approach is to combine similar functionalities of all applications.
- This properties or functions which occur repeatedly can be defined and implemented as classes for all applications.
- An application is defined only through a binding of this class.

Application-Generic Metaphor as CLasses

Example: **MediaPlayer Class**

Metaphor: A media player that can play, pause, and stop media.

Reusability: Can be used in any application that involves media playback (e.g., music player, video player).

```
class MediaPlayer:  
    def play(self):  
        print("Playing media...")  
  
    def pause(self):  
        print("Pausing media...")  
  
    def stop(self):  
        print("Stopping media...")
```

Combining Generic and Specific Metaphors

Example: Karaoke Application

```
# Generic MediaPlayer class
class MediaPlayer:
    def play(self):
        print("Playing media...")

    def pause(self):
        print("Pausing media...")

    def stop(self):
        print("Stopping media...")

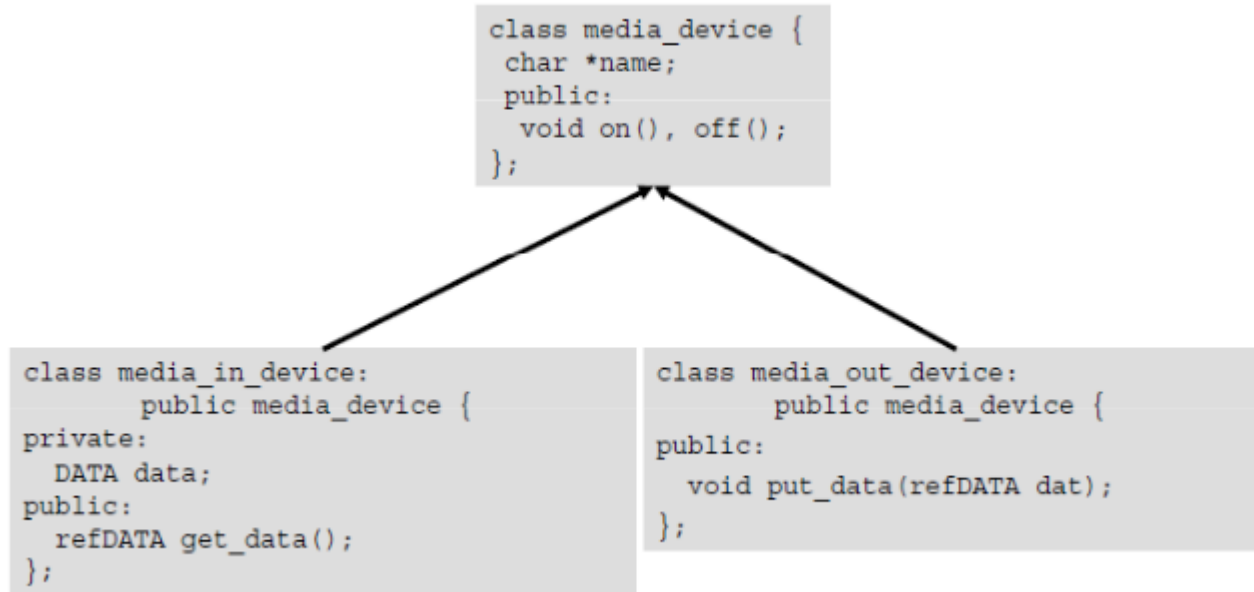
# Application-specific KaraokePlayer class
class KaraokePlayer(MediaPlayer):
    def display_lyrics(self, lyrics):
        print(f"Displaying lyrics: {lyrics}")

# Using the classes
player = KaraokePlayer()
player.play()
player.display_lyrics("Let it be...")
```

Devices as Class

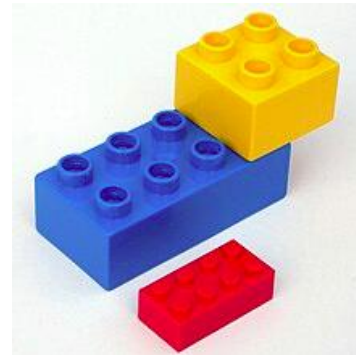
- Devices (e.g., microphones, speakers, cameras, displays) can be modeled as classes in an object-oriented design.
- This approach allows developers to encapsulate the properties and behaviors of each device into reusable and modular components.

Devices as Class



Processing Unit as a Class

- In a multimedia system can be extended to create a dataflow architecture, where source objects, destination objects, and intermediate processing objects are connected to form a pipeline for processing continuous media data (e.g., audio, video).
- This approach is often referred to as a "Lego system" because it allows developers to connect modular components (objects) to create custom dataflow paths.



Processing Unit as a Class

Components of DataFlow architecture

Source Objects: (e.g., a microphone for audio, a camera for video).

Destination Objects: (e.g., a speaker for audio, a display for video).

Intermediate Processing Objects: (e.g., audio filters, video encoders).

Channel Objects:

Act as connectors between objects, facilitating the flow of data from one object to another.

Distribution of BMOs and CMOs

- The distribution of BMOS (Basic Multimedia Objects) and CMOS (Complex Multimedia Objects) in a multimedia system refers to how these objects are organized, managed, and distributed across the system.
- BMOs: Simple, atomic multimedia objects that represent fundamental units of media (e.g., an audio sample, a video frame, or an image pixel).
- CMOs: Composite objects that are built from multiple BMOS or other CMOS (e.g., a video clip, an audio track, or a multimedia presentation).

Distribution of BMOs and CMOs

- By choosing the right distribution strategy (e.g., centralized, distributed, hierarchical, or peer-to-peer), developers can ensure that their system meets the requirements of the application while maintaining performance, scalability, and fault tolerance.
- Example: Platforms like BitTorrent use peer-to-peer distribution to share and stream media files (CMOS) composed of smaller chunks (BMOS).